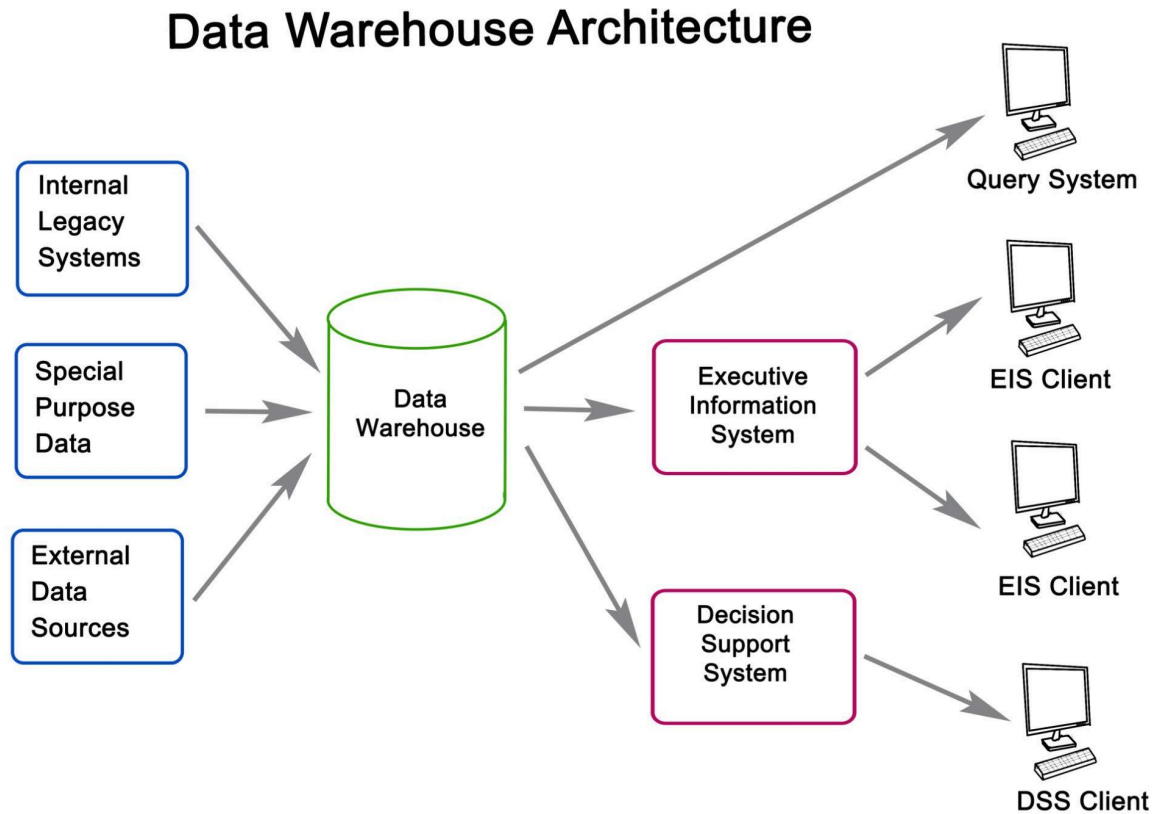


1. What is a distributed db? What are the pros and cons of it?

Definition: A distributed database (DDB) is a logically interrelated collection of databases distributed over a computer network, functioning as a single system to the end-user.



Pros:

- Reliability & Availability: No single point of failure. If one site goes down, the other sites can continue to operate.
- Performance: Faster query execution because data is located near the site of greatest demand (local access), reducing network traffic.
- Scalability: It is easier to add new nodes/sites or upgrade local hardware without disrupting the entire system.
- Local Autonomy: Each branch or site has local control and management over its own data.

Cons:

- Complexity: Designing distributed schemas, developing the DDBMS, and debugging are significantly more difficult.

- Cost: Higher expenses for hardware, network infrastructure, and personnel required to maintain the system.
 - Security: Increased attack surface because data must be continuously transmitted over telecommunication networks.
 - Concurrency & Integrity: It is highly challenging to maintain data consistency and ACID properties when a transaction requires updating data across multiple sites simultaneously.
-

2. Give an example of ddb, draw a network. Give 2 applications. Give a HF of each site.

Example & Network: An account management system for a bank with 3 branches (CN01, CN02, CN03) located in 3 different cities, connected via a Wide Area Network (WAN).

2 Applications:

1. Banking/Financial Systems: Distributing customer, account, and transaction data across local branches while maintaining a central site for consolidated reporting.
2. E-commerce/Retail: Distributing inventory and order data based on geographic regions (e.g., North Warehouse, South Warehouse) to optimize web page load times and order processing.

Horizontal Fragmentation (HF) of each site: Using the banking example above, the CUSTOMER table is horizontally fragmented based on the BRANCH attribute:

- Site 1 (Branch 1): `HF_1 = SELECT * FROM CUSTOMER WHERE BRANCH = 'CN01'`
 - Site 2 (Branch 2): `HF_2 = SELECT * FROM CUSTOMER WHERE BRANCH = 'CN02'`
 - Site 3 (Branch 3): `HF_3 = SELECT * FROM CUSTOMER WHERE BRANCH = 'CN03'`
-

3. Derived HF and distributed optimization

Derived Horizontal Fragmentation (Derived HF):

- Definition: A technique where a table (child table) is fragmented based on the fragmentation criteria of another table (parent table) through a Foreign Key relationship.
- Purpose: To ensure that logically related data is physically stored at the same site. This completely eliminates the need to JOIN two tables across the network, which heavily optimizes performance.
- Example: The CUSTOMER table (parent) is fragmented by BRANCH. The TRANSACTIONS table (child), which stores transaction history, does not have a BRANCH column; it is fragmented by "following" the CUSTOMER table. A transaction made by a customer belonging to CN01 will automatically be stored at Site 1.

Distributed Optimization:

- **Definition:** The process by which a Distributed Database Management System (DDBMS) analyzes a query and determines an Execution Plan with the lowest possible cost.
- **Core Objective:** In a distributed environment, network transmission is the most expensive factor. Therefore, the primary goal of optimization is to minimize the amount of data transmitted over the network (Communication Cost), before factoring in local I/O and CPU costs.
- **Basic Optimization Principles:**
 1. Perform Selection and Projection operations at local sites as early as possible to filter out unnecessary data before transmitting it over the network.
 2. Optimize JOIN order: JOIN smaller tables first, and larger tables later.
 3. Utilize Semi-Join techniques instead of traditional Joins to reduce the size of intermediate tables that must be transmitted between sites.

1. Primary vs. Derived Horizontal Fragmentation

To understand why we need DHF, we must first distinguish it from Primary HF:

- **Primary Horizontal Fragmentation (PHF):** Partitions a table based on predicates (conditions) applied directly to its own columns.
 - *Example:* Fragmenting the **FACULTY** table where **Department = 'CS'**.
- **Derived Horizontal Fragmentation (DHF):** Partitions a "child" table based on the predicates defined on a "parent" table.
 - *Mechanism:* This relationship is established through a **Primary Key - Foreign Key** pair.
 - *Logical Formula:* $\$Fragment(Child)_i = Child \setminus \times Fragment(Parent)_i$.

2. Why do we need Derived HF?

The most critical reason for using DHF is **Distributed Join Optimization**.

A. Avoiding Data Shuffling (Network Traffic)

If a parent table (e.g., **FACULTY**) is fragmented by location, but the child table (e.g., **CLASS**) is not, a query joining them would require the system to pull data from multiple remote nodes to a single site to perform the join.

- **With DHF:** By fragmenting the **CLASS** table "following" the **FACULTY** table, the classes taught by a specific faculty member are physically stored at the same site as that faculty member.
- **Result:** The join is performed as a **Local Join** at each site, minimizing network bandwidth usage and latency.

B. Ensuring Integrity and Mapping

DHF ensures that related fragments (like **INVOICE** and **INVOICE_DETAIL**) always stay "together". Without DHF, managing dependent records across different sites becomes extremely complex during updates or **Cascade Deletes**.

3. Practical Example: Library Management System

Imagine a library system distributed across two sites:

- 1. **Relation **BOOK** (Primary HF):** Fragmented by **BookType**. Technical books are at **Site A**, and Economics books are at **Site B**.
- 2. **Relation **LOAN_SLIP**:** This table does not have a **BookType** column.

To ensure that the loan records for technical books are also stored at **Site A** for easy auditing:

- We perform **Derived HF** on **LOAN_SLIP** based on its foreign key relationship with **BOOK**.
- **Effect:** \$LOAN_SLIP_A = LOAN_SLIP \times BOOK_A\$ (Technical books fragment).

Feature	Primary HF	Derived HF
Criteria	Predicates on own columns	Predicates on parent columns
Key Use	Optimizes single-table queries	Optimizes performance for parent-child table interactions
Main Goal	Data Locality	Local Join Optimization